

Thara, end to end

Thara is Femi9's opt-in referral and loyalty club. A customer joins free, places one qualifying order, and from then on earns three stacked benefits: a standing discount on her own orders, store credit every time a friend buys through her link, and a quarterly Amazon voucher on top.

SINGLE LEVEL · NO MLM CASCADE

FLAG: THARA_ENABLED

ENROLMENT → VOUCHERS ALL SHIPPED

THE OFFER

Three benefits, one qualifying order

Everything below unlocks off a single ₹3,000 order. The friends she refers never have to join anything — they just shop.

BENEFIT 1

Her own discount

10–20%

off her own orders, by slab

Applies to the member's own cart once she is unlocked. Slab is picked off the subtotal, before shipping.

BENEFIT 2

Referral commission

10%

of a friend's order, as Femi9 money

Store credit, applied automatically at her next checkout. No minimum on the friend's order — a ₹500 order earns ₹50.

BENEFIT 3

Quarterly Amazon voucher

1% × 3

points, tripled into rupees

1% of each friend's order becomes points. At the end of the quarter, points × 3 is issued as an Amazon gift code.

SAY THIS OUT LOUD IN THE DEMO

Benefits 2 and 3 stack on the same order. One friend spending ₹3,000 pays the member ₹300 in Femi9 money *and* 30 points that become a ₹90 Amazon voucher. They are two separate ledgers, not one pot split two ways.

Join, unlock, share

This ordering is the whole product. Every support question that has ever come in lands on step two, so give it the most airtime.

STEP 01

Join

Free, one tap, accepts the current terms version. Her referral code is minted immediately — she does not have to buy first to get a link.

STEP 02

Unlock

One order of **₹3,000 or more** switches earning on. That single order is what unlocks her — it pays full price itself.

STEP 03

Share & earn

She sends her link. Every friend who buys through it pays her commission and points, for as long as she stays active.

THE TWO OBJECTIONS THAT ALWAYS COME UP

It is one single order, not a running total. Two ₹1,500 orders never unlock the programme. The unlock meter on `/thara` plots her *biggest single paid order* against the bar and says so in words, precisely because this is the number one support ticket.

The qualifying order pays full price. Activation runs inside the same transaction that marks the order paid, but the discount is computed earlier — at checkout, when the order is created. So the first big order unlocks; the discount starts on the one after it.

The five states a member can be in

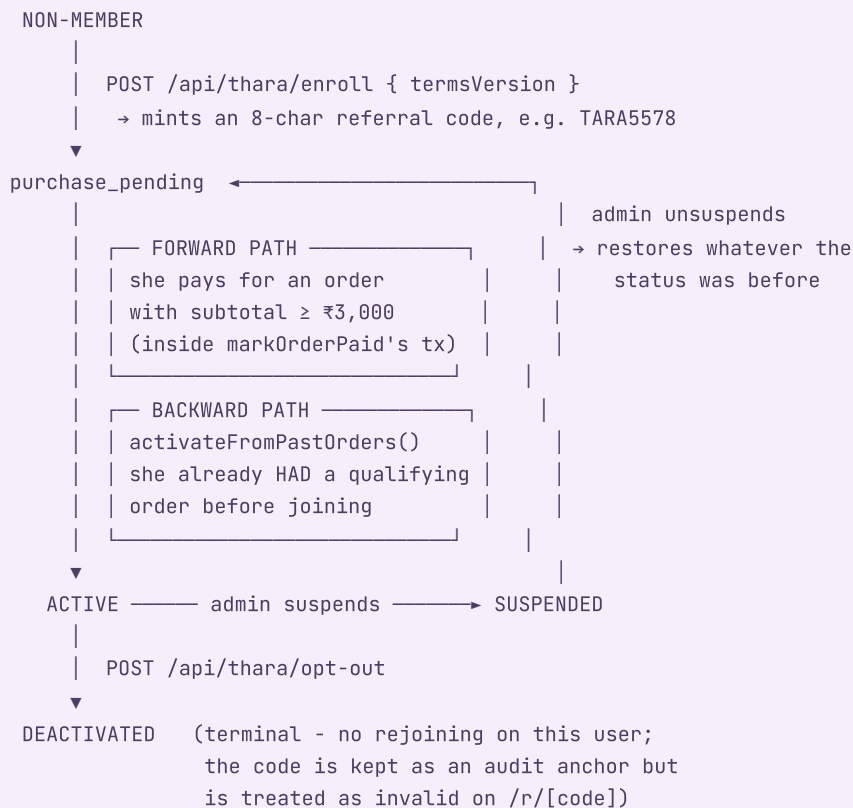
INTERNAL STATUS	WHAT SHE SEES	WHAT IT MEANS
—	no label	Not joined. The page teaches the programme and ends in a Join button.
<code>purchase_pending</code>	Joined - not unlocked yet	She's in and her link works, but she earns nothing until her own ₹3,000 order lands. The unlock meter shows here.
<code>active</code>	Unlocked - you are earning	Discount live, commission accruing, points accruing.
<code>suspended</code>	Paused by our team	Admin breaker for suspected abuse. No new accruals; everything already earned is preserved. Unsuspend restores the prior status.
<code>deactivated</code>	You left the programme	Terminal. She cannot rejoin on the same account. Earned credit stays, issued vouchers stay claimable until their deadline.

The customer never sees the left column. `purchase_pending` is a database word and rendering it to a shopper is treated as a bug.

MECHANISM

How unlocking actually works

Two paths lead to `active`, and the second one is the one people forget exists.



Activation is server-authoritative

The promotion `purchase_pending → active` runs inside the same database transaction as `markOrderPaid`. A member can never end up active with no qualifying order behind her, and the qualifying order id is stamped on the membership row in that same transaction.

Activation also looks backwards

The normal sequence is the reverse of the diagram — she buys, then finds the programme from her account page. `activateFromPastOrders` asks the mirror question and promotes off her order history, stamping the **oldest** qualifying order. It runs on enrol and on every load of `/api/thara/me` and `/summary`, so a member stuck in pending repairs herself the next time she opens the page.

ATTRIBUTION

The referral link, click to lock

Codes are **4 letters + 4 digits**, drawn from alphabets that exclude **I**, **L**, **O**, **0** and **1** — because these get read aloud across a table and typed by hand. About 1.15 billion possible codes.

T A R A 5 5 7 8

Friend gets `https://femi9.in/r/TARA5578` (WhatsApp, SMS, Instagram bio)

|
▼ GET /r/[code]

1. Feature flag off?	→ 404
2. Code malformed?	→ 302 /
3. Code not found?	→ 302 /
4. Referrer suspended/left?	→ 302 / (no cookie)
5. Otherwise: set signed HttpOnly cookie	
femi9_thara_ref = HS256 jwt(membershipId)	
aud 'femi9-thara-ref', TTL 30 days	
	→ 302 /

|
▼ she browses normally, then signs up

attributeReferralIfPresent()
runs on ALL THREE sign-in paths:
phone OTP · email magic link · Google OAuth
fire-and-forget - attribution never blocks sign-in
reject if: no cookie · bad signature · referrer gone
· referrer suspended/left · SAME USER
· same verified email · same phone
· already attributed to someone else
otherwise: create TharaReferral, stamp IP + UA
clear the cookie either way

|
▼ friend's first paid order ≥ ₹3,000
TharaReferral.lockedAt = now ← permanent from this instant

THE WINDOW THAT MATTERS

Between signup and the friend's first qualifying order, the referral row exists with `lockedAt = null`. That is the window in which an admin can delete an obviously fraudulent attribution. After the lock it is permanent, and there is deliberately **no admin API to attach a referrer after the fact** — the "I forgot to use the link, please credit me" flow does not exist by design.

Note that a `purchase_pending` referrer's link *does* still set the cookie and capture attributions. She simply earns nothing on them until she unlocks herself, and past downline orders do not backfill once she does.

THE NUMBERS

What the money looks like

Lead the demo with this panel. Abstract percentages don't land — rupees do.

WORKED EXAMPLE · ONE FRIEND, ONE ORDER

Your friend spends	₹3,000
You get Femi9 money (10%)	₹300
You also collect points (1%)	30 points
Those points become a voucher (× 3)	₹90 Amazon
Total value from that one order	₹390

That is one friend, one order. Ten friends doing the same is ten times as much.

Her own discount slabs

ORDER SUBTOTAL	DISCOUNT	ON A CART OF...	SHE PAYS
under ₹3,000	none	₹2,900	₹2,900
₹3,000 – ₹5,999	10%	₹4,000	₹3,600
₹6,000 – ₹8,999	15%	₹7,000	₹5,950
₹9,000 and above	20%	₹12,000	₹9,600

Discount is computed on **subtotal, before shipping**, and floored so the slab is never rounded up. It is applied server-side at order creation — whatever the client shows is a preview; the authority is `Order.discount`.

Two ledgers, both append-only

Femi9 money — TharaCreditLedger

Every commission is a positive `delta`; every spend at checkout is a negative one; every refund reversal is a mirror-signed row referencing the original. Rows are never edited. Balance is `SUM(delta)`, clamped to ≥ 0 for spending, and each row also snapshots `balanceAfter` for audit.

Spent automatically at her next checkout, debited inside the same transaction as the order so two concurrent checkouts cannot double-spend one balance.

Points — TharaRewardPointsLedger

Scoped to a `TharaCycle` — fixed calendar quarters, Jan–Mar, Apr–Jun, Jul–Sep, Oct–Dec. Points are **not spendable**; they only become value at cycle close, when `closeCycle()` sums each user's points, issues a `TharaVoucher` at points $\times 3$, and stamps a 30-day claim deadline.

Close is idempotent — a second call on the same cycle is a no-op, guarded by a unique index on `(userId, cycleId)`.

NEVER SAY "WALLET BALANCE YOU CAN WITHDRAW"

Femi9 money is **store credit only** — not withdrawable to bank or UPI, not transferable, and it never expires. The Amazon voucher is the only thing that leaves the Femi9 economy, and it expires 30 days after issue.

GUARD RAILS

Refunds and abuse

When a friend returns an order

Refunding an order writes mirror-signed reversal rows on both ledgers, inside the refund's own transaction. The commission earned by the referrer is clawed back, and any Thara credit the buyer spent on that order is given back to her — so a refund is whole from both sides. Because `refundOrder` gates on `status == 'paid'`, it runs at most once per order.

Negative balances are allowed. If she already spent the credit before the refund landed, her balance goes negative and she cannot spend again until fresh earnings restore it to zero.

Fraud guards, cheapest first

GUARD	ENFORCED WHERE	BEHAVIOUR
Self-referral	signup hook	Friend's user id equals the referrer's → rejected silently.
Duplicate account — email	signup hook	Friend's verified email matches the referrer's → rejected.
Duplicate account — phone	signup hook	Friend's phone matches the referrer's → rejected.
Suspended referrer	<code>/r/[code]</code> + signup hook	No cookie set; any existing cookie is ignored at signup.
One referrer per user, forever	DB unique index	<code>TharaReferral.referredUserId</code> is unique — a second link simply won't insert.
Cookie unforgeability	HS256 over <code>AUTH_SECRET</code>	Distinct audience <code>femi9-thara-ref</code> ; hand-crafted cookies fail verification.
No post-hoc attribution	by omission	There is no admin route to attach a referrer to an existing account.
IP / User-Agent capture	<code>TharaReferral</code>	Non-blocking. Powers "this referrer's 50 signups share one subnet" queries.
Invite rate limits	<code>/api/thara/invite</code>	10 per user per hour, 20 per IP per hour.
Admin suspend	admin console	Ops breaker. Preserves the prior status so unsuspend is clean.

REALITY CHECK

What is actually built

Verified against the code today, not against the status table in `docs/thara/PROGRAM.md` — that table is stale and still describes B through F as unbuilt.

SUB-PROJECT	STATUS	EVIDENCE IN THE CODE
A · Enrolment + attribution	SHIPPED	Schema, flag, code minting, signed cookie, all three sign-in hooks, activate-and-lock in <code>markOrderPaid</code> .
B · Personal discount	SHIPPED	<code>computeTharaDiscount()</code> , wired into <code>placeOrder</code> at <code>checkout.ts:331</code> . 9 integration tests.
C · Wallet + commission	SHIPPED	<code>TharaCreditLedger</code> , <code>accrueTharaCommission</code> , <code>applyTharaCredit</code> , refund reversal.
D · Points + vouchers	SHIPPED, MANUAL CODES	Cycles, points ledger, <code>closeCycle</code> , expiry cron, claim route. The Amazon Incentives issuer is a stub that throws — it falls through to <code>ManualIssuer</code> and an admin pastes codes.
E · Email invites	SHIPPED	Resend delivery, suppression list, self-invite and bad-address guards, rate limits.
F · Dashboards	SHIPPED	<code>/thara</code> (856 lines) and <code>/admin/thara</code> (513 lines), both drawn to the design-system comp.

TEST COVERAGE, VERIFIED JUST NOW

84 Thara tests pass — 15 unit (codes, cookies, feature flag) and 69 integration across enrolment, attribution, activation, backfill, discount, credit, rewards, invites and admin. If you run the full suite you will see Thara files fail: that is a Postgres deadlock in the shared `TRUNCATE` reset when files run in parallel against the remote test database, not a logic failure. Run them with `--no-file-parallelism` and they are green.

Three things to know before you open your mouth

1 · COUPONS STACK WITH THE THARA DISCOUNT IN CODE

Both `PROGRAM.md §5` and the UI brief promise "no stacking — the system applies whichever saves her more." **The checkout does not do that.** At `checkout.ts:331-368` the Thara slab discount is computed first, then a coupon is added on top, capped only at the remaining subtotal. A 20% member with a 20% coupon currently gets both. No test covers the interaction. Decide before the demo whether to show a coupon at all — and if someone asks, this is a known gap, not a misunderstanding.

2 · PARTIAL REFUNDS ARE SPEC, NOT CODE

The docs describe proportional clawback on a partial refund. The implementation has one refund path and it is all-or-nothing: `refundOrder` reverses every ledger row tied to the order in full. There is no partial-refund entry point yet.

3 · AMAZON CODES ARE PASTED BY HAND

`AmazonIncentivesIssuer` throws `NotImplementedError` by design — the real `CreateGiftCard` call still needs credentials and SigV4 signing, and Amazon's onboarding review takes two to four weeks. Until then `closeCycle` catches that, falls back to `ManualIssuer`, and the voucher is created with `amazonCode = null`. It surfaces in the admin console's "**Voucher queue - needs Amazon code**" panel. That fallback is worth demoing on purpose — it shows the programme runs today without waiting on Amazon.

The demo script

Roughly 12 minutes. Have two browser profiles ready — one signed in as **Asha** (the member), one clean for **Bela** (the friend) — plus an admin tab. Confirm

`THARA_ENABLED=true` before you start; every Thara route 404s when it's off.

01 Open `/thara` as a signed-in non-member

This is the join screen. Point out that the explainer comes *before* any status — the page teaches first and reports second.

Say: "Every rupee figure on this page comes from the server at runtime, not from copy typed into the template. If the business retunes a slab tomorrow, this page cannot start lying about it."

02 If the account already has a big order, show the conversion moment

The join screen tells an already-qualified shopper she'll unlock the instant she joins — "your ₹4,000 order already meets the ₹3,000 rule."

Say: "She doesn't have to buy again. We look backwards through her order history at the moment she joins."

03 Join, and land on the unlock meter

She's now `purchase_pending` — but the label reads "Joined - not unlocked yet." The meter plots her biggest single paid order against ₹3,000.

Say: "Read the copy under the meter. Orders are not added together. This one sentence is here because it was our single biggest source of support tickets."

04 Copy the referral link

Show the code, the copy button's confirmed state, and the full URL as selectable text below it. Mention the alphabet: no `I`, `L`, `0`, `0` or `1`, because people read these aloud.

05 Switch to the clean profile and open the link

It 302s to the homepage and looks like nothing happened. Open devtools and show `femi9_thara_ref` in the cookie jar — HttpOnly, 30 days.

Say: "That cookie is a signed JWT. A shopper can't hand-craft one to credit whoever she likes."

06 Sign up as Bela and place an order over ₹3,000

The attribution row is written silently at signup. The lock lands when the order is paid.

Say: "Bela never joined anything. She's a normal customer. That's deliberate — the friend's experience is unchanged."

07 Back to Asha's /thara

Her friend count is now 1, her Femi9 money balance shows the 10% commission, and the points-this-cycle block shows 1% with the estimated voucher value beside it.

Say: "Two separate ledgers from one order. The money is spendable now; the points only become real at the end of the quarter."

08 Put something in Asha's cart and go to checkout

The slab discount and the credit apply together — discount first on the subtotal, then the wallet balance comes off the remaining total. Both land server-side inside one transaction.

Say: "No coupon code, no opt-in. Her credit just comes off the bill."

09 Open the admin console at /admin/thara

Headline metrics — members by status, referrals total vs locked, commission accrued vs spent, points, voucher counts. Then search the member table by referral code or email and show the suspend control with its reason field.

10 Force a cycle close

POST /api/cron/thara-close-cycle — admin session or x-cron-secret. Points are summed per user, vouchers issued at points × 3, and the cycle is stamped closed. The new voucher lands in the "needs Amazon code" queue; paste a code and show it turn claimable on Asha's page.

Say: "Amazon's API onboarding is still in review, so we ship with a manual issuer behind the same interface. The day the credentials land it's an env-var change, not a rewrite."

11 Close on the refund, if you have time

Refund Bela's order from the admin order view and show both ledgers grow a mirror-signed reversal row. Nothing is edited or deleted.

Say: "Append-only. We can always reconstruct exactly what a member's balance was on any date, and why."

Questions you will get

Do the friends have to join Thara too?

No. They shop normally. Enrolment is only what unlocks *earning*, and only for the referrer.

Is this an MLM? Do I earn on my friend's friends?

No. It is strictly single-level. A refers B; A earns on B's orders. If B then refers D, B earns on D — A gets nothing from D. There is no cascade and nothing in the schema could support one.

I placed two orders. Why am I still locked?

The rule measures a single order, not a running total. Two ₹1,500 orders never unlock it; one ₹3,000 order does. The unlock meter deliberately plots the biggest single paid order and says orders are not added together.

Does my first big order get the discount?

No. Activation happens inside the transaction that marks that order paid, but the discount was computed earlier, at checkout. The qualifying order pays full price and unlocks every order after it.

What if someone enrolls and never buys anything?

They sit in `purchase_pending` indefinitely. Their link still works and still captures attributions — but when the referred friend pays, only the friend's own lock happens. The referrer earns nothing. If she later completes her own ₹3,000 order she becomes active, and *future* downline orders start earning. Past ones do not backfill.

Can a member share the link publicly — Instagram bio, WhatsApp status?

Yes, there is no per-code rate limit on `/r/[code]`. The email invite path is rate-limited (10/user/hour, 20/IP/hour), and the IP and User-Agent captured at signup let ops spot link-farming clusters and suspend manually.

What happens to a suspended member's earnings?

Everything already accrued stays in the ledger, and issued vouchers stay valid until their claim deadline. Only *new* commission and points stop. Unsuspending restores whatever status she held before, via `statusBeforeSuspend`.

Can someone be both a member and someone else's referral?

Yes — the two are independent. She earns on her own downline's orders, and her referrer earns on hers.

Is Femi9 money real money?

It comes off her next Femi9 order and nothing else. Not withdrawable to bank or UPI, not transferable, no expiry. The Amazon voucher is a real gift code, and that one does expire — 30 days from issue.

Where everything lives

Business logic

`src/lib/services/thara.ts`

918 lines — every rule: enrol, attribute, activate, discount, commission, points, cycles, vouchers, invites, admin

`src/lib/thara/codes.ts` code generation and normalisation, with the ambiguity-free alphabets

`src/lib/thara/cookies.ts` signs and verifies `femi9_thara_ref`

`src/lib/thara/voucher-issuer.ts` the `VoucherIssuer` interface, plus the Amazon and manual implementations

`src/lib/thara/invite.ts` Resend delivery, mirrors `otp.ts` so a swap to SES is one file

`src/lib/thara/feature.ts` `isTharaEnabled()`, read at call time so a flipped env needs no code change

Integration points

`checkout.ts:331` `computeTharaDiscount` at order creation

`checkout.ts:446` `applyTharaCredit` debits the wallet in the same tx

`checkout.ts:685-694` activate + lock, commission, points — all inside `markOrderPaid`

`admin/orders.ts:410-413` both reversals inside `refundOrder`

`services/auth.ts` `verify0tp`, `verifyMagicLink`, `signInWithGoogle` each take an attribution context

Routes

`/r/[code]` public referral link — sets the cookie, 302s home

`/thara` · `/admin/thara` customer page and admin console

`/api/thara/*` `enroll`, `me`, `summary`, `invite`, `opt-out`, `vouchers/[id]/claim`

`/api/admin/thara/*` `list`, `metrics`, `[id]`, `[id]/suspend`, `[id]/unsuspend`, `vouchers`, `vouchers/[id]/set-code`

`/api/cron/thara-close-cycle` quarterly, EventBridge or admin. Sums points, issues vouchers, stamps the cycle closed

`/api/cron/thara-expire-vouchers` batch-expires past-deadline vouchers

Data model

TharaMembership	one per user. Status, referral code, terms version, qualifying order, suspend bookkeeping
TharaReferral	one per referred user (unique). Attribution time, lock time, IP + UA
TharaCreditLedger	append-only paise. <code>+earn / -spend / ±reversal</code> , with a <code>balanceAfter</code> snapshot
TharaRewardPointsLedger	append-only points, scoped to a cycle
TharaCycle	· <code>TharaVoucher</code> quarter windows and the vouchers they close into
TharaSuppressedEmail	invite suppression on bounce or complaint

Tunable defaults

SETTING	DEFAULT	WHERE
Qualifying minimum	₹3,000	THARA_QUALIFYING_MIN_PAISE
Commission rate	10%	THARA_COMMISSION_PCT
Points rate	1%	THARA_POINTS_PCT
Voucher multiplier	× 3	THARA_VOUCHER_MULTIPLIER
Claim window	30 days	THARA_VOUCHER_CLAIM_DAYS
Discount slabs	10 / 15 / 20%	TharaDiscountSlabs
Terms version	v1	THARA_TERMS_VERSION
Feature flag	off	THARA_ENABLED
Voucher issuer	manual	THARA_VOUCHER_ISSUER

All of these are constants in `src/lib/services/thara.ts` today and are echoed to the customer page through `/api/thara/summary`'s `rules` block — which is why no number is ever hardcoded in the UI. The original design called for a `TharaSetting` table so the business team could retune without a deploy; that table does not exist yet.

Compiled from the Femi9 codebase on branch `master`. Sources: `docs/thara/PROGRAM.md`, `docs/thara/THARA_UI_E2E.md`, `src/lib/services/thara.ts`, `src/lib/services/checkout.ts`, `prisma/schema.prisma`. Where the docs and the code disagreed, the code won and the disagreement is flagged above.